BFL: End-to-End Toy Workflow

```
library(BFL)
set.seed(1)
```

Overview

This vignette demonstrates a complete workflow using only functions exported by {BFL}:

1. Generate a fully-labeled toy VA-like dataset
2. Use make_label_mask() to derive the observed-label structure for: Base, Domain, Partial, Mix
3. (Optional) Train/predict local LCVA models: • single_lcva() • multi_lcva()
4. Run global aggregation with run_BFL()
5. Sample posterior predictive draws with predict_BFL()
6. Evaluate with score_BFL() and interpret metrics (Top-1 + CSMF)

See the package manual for function definitions and return objects.

Generating a toy dataset

We simulate binary symptoms X and categorical causes Y. This is not a realistic VA generative model. It's a lightweight scaffold for demonstrating the API.

```
simulate_va <- function(N, P, causes = c("A","B","C"), p_symptom = 0.25) {
  X <- matrix(rbinom(N * P, 1, p_symptom), nrow = N, ncol = P)
  Y <- sample(causes, N, replace = TRUE)
  list(X = X, Y = Y)
}

P <- 15

# training data (fully labeled)
train <- simulate_va(N = 150, P = P, p_symptom = 0.25)

# target data (fully labeled truth, which we will partially hide)
target <- simulate_va(N = 100, P = P, p_symptom = 0.30)

X_train <- train$X
Y_train <- train$Y

X_target <- target$X
Y_true   <- target$Y
```

```r
c(N_train = nrow(X_train), N_target = nrow(X_target), P = ncol(X_train))
#>  N_train N_target        P
#>      150      100       15
table(Y_true)
#> Y_true
#>  A  B  C
#> 22 47 31
```

Optional: make_label_mask() — turning full labels into regimes

In simulations we have full labels (Y_true). But BFL regimes differ by which labels are observed vs hidden.

make_label_mask() does this for you: it masks labels (sets some to NA) and returns the index sets needed by different variants (Base/Domain/Partial/Mix).

The important outputs are: • Y_obs: observed label vector with NA for hidden labels

- missing_idx: rows treated as unlabeled
- stan_idx: rows that Stan uses
- plus regime-specific sets (labeled_idx, domain_idx, partial_idx) for some models

```r
miss_prop <- 0.80  # 80% unlabeled => 20% labeled

mask_base <- make_label_mask(Y_true, model = "Base", miss_prop = miss_prop, seed = 1)
mask_dom  <- make_label_mask(Y_true, model = "Domain", miss_prop = miss_prop, seed = 1)
mask_par  <- make_label_mask(Y_true, model = "Partial", miss_prop = miss_prop, seed = 1)
mask_mix  <- make_label_mask(
  Y_true,
  model = "Mix",
  miss_prop = miss_prop,
  mix_domain_prop_within_labeled = 0.50,
  seed = 1
)

c(
  base_labeled    = sum(!is.na(mask_base$Y_obs)),
  domain_labeled  = sum(!is.na(mask_dom$Y_obs)),
  partial_labeled = sum(!is.na(mask_par$Y_obs)),
  mix_labeled     = sum(!is.na(mask_mix$Y_obs))
)
#>    base_labeled  domain_labeled partial_labeled
#>               0              20              20
#>     mix_labeled
#>              20
```

How the dataset differs across Base / Domain / Partial / Mix

The same target dataset becomes different "observed-label views".

```r
preview <- data.frame(
  Y_true  = Y_true,
  Base    = mask_base$Y_obs,
  Domain  = mask_dom$Y_obs,
  Partial = mask_par$Y_obs,
  Mix     = mask_mix$Y_obs
)

head(preview, 12)
#>    Y_true Base Domain Partial  Mix
#> 1       B <NA>   <NA>    <NA> <NA>
#> 2       A <NA>   <NA>    <NA> <NA>
#> 3       A <NA>      A       A    A
#> 4       C <NA>      C       C    C
#> 5       B <NA>      B       B    B
#> 6       C <NA>   <NA>    <NA> <NA>
#> 7       B <NA>   <NA>    <NA> <NA>
#> 8       B <NA>      B       B    B
#> 9       B <NA>   <NA>    <NA> <NA>
#> 10      C <NA>      C       C    C
#> 11      B <NA>   <NA>    <NA> <NA>
#> 12      C <NA>   <NA>    <NA> <NA>

counts <- rbind(
  Base    = c(labeled = sum(!is.na(mask_base$Y_obs)),
              missing = length(mask_base$missing_idx), stan = length(mask_base$stan_idx)),
  Domain  = c(labeled = sum(!is.na(mask_dom$Y_obs)),
              missing = length(mask_dom$missing_idx),  stan = length(mask_dom$stan_idx)),
  Partial = c(labeled = sum(!is.na(mask_par$Y_obs)),
              missing = length(mask_par$missing_idx),  stan = length(mask_par$stan_idx)),
  Mix     = c(labeled = sum(!is.na(mask_mix$Y_obs)),
              missing = length(mask_mix$missing_idx),  stan = length(mask_mix$stan_idx))
)
counts
#>         labeled missing stan
#> Base          0     100  100
#> Domain       20      80   80
#> Partial      20      80  100
#> Mix          20      80   90
```

Interpretation (high level):

- Base: no labels observed; Stan uses all rows
- Domain: Stan sees only unlabeled rows (stan_idx == missing_idx)
- Partial: Stan sees all rows; labels are partially missing
- Mix: like Partial, but a labeled "domain" subset is excluded from Stan

Optional LCVA training (local model summaries)

run_BFL() aggregates local summaries, where each local model provides an N × C_m score matrix over the same target records.

The cleanest way to produce a local summary in this package is:

- fit_lcva() then predict_lcva() (or the wrappers single_lcva(), multi_lcva())

This section runs only if {LCVA} is available.

```
library(LCVA)
has_lcva <- requireNamespace("LCVA", quietly = TRUE)
has_lcva
#> [1] TRUE
```

One-shot LCVA

single_lcva() fits an LCVA model on training data and immediately predicts on a target dataset.

It:

1. Fits LCVA on (X_train, Y_train).

2. Computes posterior cause scores for X_target.

3. Returns per-record posterior means (posterior_phi), estimated target prevalence (pi_pred), top-1 predictions (Y_pred), and row-alignment metadata.

```
# One-shot LCVA: fit + predict
lcva_one <- single_lcva(
  X_train, Y_train, X_target,
  lcva_args = list(Nitr = 200, thin = 2, seed = 1, pred_Nitr = 400)
)

names(lcva_one)
#> [1] "posterior_phi" "cause_ids"     "pi_pred"
#> [4] "Y_pred"        "target_info"
dim(lcva_one$posterior_phi)  # N x C
#> [1] 100   3
head(lcva_one$pi_pred)
#> [1] 0.3551451 0.2103058 0.4345491
head(lcva_one$Y_pred)
#> [1] C C C C C A
#> Levels: A B C
```

Multi LCVA

multi_lcva() fits a single LCVA model on the training data and applies it to multiple target datasets.

It:

1. Fits LCVA once on (X_train, Y_train).

2. Predicts on each matrix in a named list of targets.

3. Returns a list of prediction summaries, each containing posterior_phi, pi_pred, Y_pred, and row-alignment metadata.

```
# Multi LCVA: fit once, predict many targets (toy)
targets <- list(
  targetA = X_target,
```

```
  targetB = simulate_va(N = 90, P = P, p_symptom = 0.35)$X,
  targetC = simulate_va(N = 70, P = P, p_symptom = 0.20)$X
)

lcva_many <- multi_lcva(
  X_train, Y_train, targets,
  lcva_args = list(Nitr = 200, thin = 2, seed = 2, pred_Nitr = 400)
)

names(lcva_many$targets)
#> [1] "targetA" "targetB" "targetC"
str(lcva_many$targets$targetA, max.level = 1)
#> List of 5
#>  $ posterior_phi: num [1:100, 1:3] 0.000161 0.000207 0.000458 0.000468 0.000482 ...
#>  $ cause_ids    : chr [1:3] "A" "B" "C"
#>  $ pi_pred      : num [1:3] 0.367 0.397 0.236
#>  $ Y_pred       : Factor w/ 3 levels "A","B","C": 2 2 3 1 2 1 2 1 1 2 ...
#>  $ target_info  :List of 3
```

Run global BFL aggregation with run_BFL()

run_BFL() expects:

- local_summaries: named list of local summary objects, each with: posterior_phi, cause_ids, and target_info$row_hash
- X_target: used to define canonical row identity via hashing
- Y_target: optional vector with NA for unknown labels (partial label setting)
- model: “Base” / “Domain” / “Partial” / “Mix” (stored in output)

Below we run Base/Domain/Partial/Mix using the observed labels from make_label_mask().

```
# Build a minimal local_summaries list from one LCVA summary.
# In practice you'd include multiple local models/sites; the API is the same.
local_summaries <- list(
  LCVA = list(
    posterior_phi = lcva_one$posterior_phi,
    cause_ids     = lcva_one$cause_ids,
    target_info   = lcva_one$target_info
  )
)

fit_base <- run_BFL(local_summaries, X_target, Y_target = mask_base$Y_obs, model = "Base")
fit_dom  <- run_BFL(local_summaries, X_target, Y_target = mask_dom$Y_obs,  model = "Domain")
fit_par  <- run_BFL(local_summaries, X_target, Y_target = mask_par$Y_obs,  model = "Partial")
fit_mix  <- run_BFL(local_summaries, X_target, Y_target = mask_mix$Y_obs,  model = "Mix")

names(fit_mix)
#> [1] "pi"          "lambda"      "phi"
#> [4] "causes"      "model_names" "row_hash"
#> [7] "model"
```

Diagnostics with eval_BFL()

It reports:

- the estimated source weights (lambda_table),
- posterior summaries of the target cause fractions (pi_summary),
- and correlations between local and aggregated cause scores (phi_correlation).

Use this function to inspect how the global model combined local summaries and how strongly each source influenced the final fit.

```
diag_mix <- eval_BFL(fit_mix)
names(diag_mix)
#> [1] "lambda_table"    "pi_summary"
#> [3] "phi_correlation"
str(diag_mix$lambda_table)
#> 'data.frame':    3 obs. of  2 variables:
#>  $ cause: chr  "A" "B" "C"
#>  $ LCVA : num  1 1 1
head(diag_mix$pi_summary)
#>         A         B         C
#> 0.1398250 0.2507868 0.6093882
```

Posterior predictive draws with predict_BFL()

predict_BFL() returns posterior predictive draws per target record:

- draws_int (S × N, integer class indices)
- draws (S × N, class labels) • pi posterior draws (S × C) • plus bookkeeping (causes, model, row_hash)

```
pred_base <- predict_BFL(fit_base, seed = 1)
pred_dom  <- predict_BFL(fit_dom,  seed = 1)
pred_par  <- predict_BFL(fit_par,  seed = 1)
pred_mix  <- predict_BFL(fit_mix,  seed = 1)

names(pred_mix)
#> [1] "draws_int" "draws"     "causes"    "pi"
#> [5] "model"     "row_hash"
dim(pred_mix$draws_int)
#> [1] 4000  100
head(pred_mix$causes)
#> [1] "A" "B" "C"
```

Scoring with score_BFL() (Top-1, CSMF, confusion, per-cause errors)

```
sc_base <- score_BFL(pred_base, mask = mask_base)
sc_dom  <- score_BFL(pred_dom,  mask = mask_dom)
```

```r
sc_par  <- score_BFL(pred_par,  mask = mask_par)
sc_mix  <- score_BFL(pred_mix,  mask = mask_mix)

out <- rbind(
  Base    = c(top1 = sc_base$top1_acc, csmf = sc_base$csmf_acc),
  Domain = c(top1 = sc_dom$top1_acc,  csmf = sc_dom$csmf_acc),
  Partial= c(top1 = sc_par$top1_acc,  csmf = sc_par$csmf_acc),
  Mix     = c(top1 = sc_mix$top1_acc,  csmf = sc_mix$csmf_acc)
)

out
#>          top1      csmf
#> Base     0.4 0.2568158
#> Domain   0.4 0.6826788
#> Partial  0.4 0.6161690
#> Mix      0.4 0.6686546
```

Metrics: what they mean (and what's being evaluated)

Two core metrics: • Top-1 accuracy: uses majority vote across posterior draws and compares to truth. (In-package helper: get_ACC(pred, true_Yt).)

- CSMF accuracy: compares prevalence vectors pi_hat vs pi_true.

Because we're using mask-based evaluation, the default interpretation is:

- evaluate Top-1 on the intended subset (often unlabeled rows)
- compute CSMF in a way consistent with the regime (Domain/Mix need stan_idx to reconstruct)

```r
# Example: top-1 from helper (uses posterior mode across draws)
acc_mix <- get_ACC(pred_mix, Y_true)
acc_mix
#> [1] 0.39

# Example: CSMF directly from pi_hat vs empirical pi_true (full truth)
pi_true <- prop.table(table(factor(Y_true, levels = pred_mix$causes)))
pi_hat  <- colMeans(pred_mix$pi)
CSMF_acc(pi_hat, as.numeric(pi_true))
#> [1] 0.616169
```

Summary

- Start from a fully labeled toy dataset.
- Use make_label_mask() to create Base/Domain/Partial/Mix observed-label structures.
- Fit local models (optional LCVA section) to produce local_summaries.
- Run global aggregation with run_BFL().
- Sample predictions with predict_BFL().
- Score consistently with score_BFL(), and interpret Top-1 + CSMF metrics.